

Design and analytical performance of a hybrid model using deep learning techniques for the detection of deepfake videos

Karan R Talwalkar
SVKM'S NMIMS
Thane, India
karantalwalkar@gmail.com

Kshitij Navale
SVKM'S NMIMS
Thane, India
kshitijnavale2703@gmail.com

Aditya Ashok
SVKM'S NMIMS
Mumbai, India
adipbmail@gmail.com

Dr. Anindita Khade
SVKM'S NMIMS
Mumbai, India
anindita.khade@nmims.edu

Abstract—Recently, it has been possible to develop utilities that can create video face-swaps that are indistinguishable from the real. Recent advancements in deep learning techniques make it possible to synthesize ever-realistic videos. Popularly referred to as deepfakes, this technology has indeed brought a lot of ease and more realistic fakes than the normal ones. In as much as deepfakes are easy to make, their detection is challenging. The study presents a hybrid model that integrates GRU in EfficientNet-B1, InceptionV3, Xception, and DenseNet121. These models were evaluated as pure networks without the GRU layers and as networks with the GRU layers positioned to process temporal information. The GRU, which is a recurrent neural network variant, serves the purpose of adding an extra edge in performance in detecting deepfakes by looking at the temporal aspect of the data. Over the course of the research, there has been an emphasis on standard models and their variation with GRU models in order to understand the role of time-based data. This study enhances constant efforts against deepfakes and will help researchers & practitioners who develop deepfake detection systems and protect the integrity of the digital media.

Keywords—Efficientnet-B1, Inception-v3, Xception, Densenet 121, GRU, Deep-Fake Video Detection.

I. INTRODUCTION

The rapid integration of digital technologies has ushered in an interesting but disturbing development whereby videos are manipulated; this is commonly known as deepfakes. Deepfake videos, a type of manipulated videos that swap faces with other images, have become more realistic and common, and present challenges to privacy, security, and the very nature of the images themselves. As more and more apps based on deep learning become available, the production of highly realistic fake videos has become easier than ever, making the average viewer unable to distinguish the difference between reality and dreaming. The reality and the role of deepfake technology have further been complemented by the development of technologies that achieve the embedding of fake footage into video images resulting in qualitatively new visual clips with manufactured scenes, but with slight imperfections that are hard to isolate without severe analytical tools. This complexity surrounding

deepfakes, in particular the need for more reliable methods of their diagnosis has become more pronounced than ever before since the innovative methods of manipulation are most often faster than the conventional methods of their detection. In light of this escalating problem, the research area of deepfake detection has matured, most notably, in the usage of Convolutional Neural Networks (CNNs). These complex models have been successful in learning the details and patterns of video manipulations. But, the response of these models differs quite greatly in terms of the particular kinds of deepfakes that they were aimed at detecting. This present research seeks to assist in the fight against deepfake technologies by investigating the performance of the four most popular deep learning model architectures: EfficientNetB1, InceptionV3, DenseNet121, and Xception. Also, we examine the opportunity of using Gated Recurrent Units (GRU) with any of the above-mentioned architectures. This study uses the Kaggle Deep Fake Detection Challenge dataset which has videos of altered images to benchmark fake image-based techniques.

The objectives of this study are as follows:

- To evaluate the efficiency of the EfficientNetB1, InceptionV3, DenseNet121, and Xception models with GRU and without GRU in the detection of deepfake videos.
- To provide the results of performed models in regards to the accuracy, precision, and efficiency of computations.
- To understand the approaches adopted by each model in the detection of deepfake videos and their respective merits and demerits.
- To assist in the discussion surrounding the applicable tactics that can be employed in the fight against deepfakes and offer suggestions on more effective strategies that can be up for examination.

Through this broad assessment, our purpose is to find the most efficient options for defending against deepfakes and improving current frameworks for detecting altered

videos. The results of this study are expected to be useful not only to computer vision and media forensics scientists but also to policymakers, content providers, and platforms facing the challenge of deepfake videos. As deepfake technology is improving, it becomes more and more evident that there is a necessity for developing efficient countermeasures. In this regard, the work presented in this study is a step in the right direction toward building stronger and more reliable systems for managing the presence of visual online content in the digital age.

II. LITERATURE REVIEW

The rapid growth of deepfake technologies has driven the search for effective detection techniques. This chapter outlines the key advancements in deepfake detection focusing on deep learning mechanisms and concerning models which are the subject of our research.

A. Evolution of Deepfake Detection

The primary analysis involving the detection of deepfakes has included computer vision operations. Li et al. [1] came up with the first method of detecting deepfakes that hopes to focus on the abnormal eye blink occurrence while watching a video. But, as it happened, with the advancement of deepfake technologies, the invention of more complicated methods of detection became necessary. Since then, deep learning methods have been at the forefront. Rossler et al. [2] in the course of developing FaceForensics++, a novel large-scale benchmark for the automatic detection of facial manipulation, evaluated several image manipulation detection CNN models. Their results revealed how powerful deep learning models could be in content manipulation detection and offered a useful tool for future work.

B. Use of Convolutional Neural Networks in Deepfake Detection

Convolutional Neural Networks (CNNs) have been widely applied to image and video tasks such as deepfake detection. In terms of application, several CNN designs have been investigated in this regard:

1) *ResNet*: The Relative contribution of Newi is due to the accessibility of deep networks. He et al. [3] presented ResNet and showed its function in image recognition tasks. In the case of deep-fake detection, Nguyen et al. [4] used a capsule network based on ResNet features and were able to classify videos with a high accuracy rate.

2) *MobileNet*: Howard et al. [5] proposed MobileNet which is an optimized aggregate depth-wise separable convolutional neural network for several mobile and embedded vision-based applications. It is efficient that one is tempted to use it in real-time deep-fake detection applications. In a study or research by Trinh et al. [6], MobileNetV2 was modified, and on-device deep-fake detection was achieved.

EfficientNet: Tan and Le [7] proposed EfficientNet which came up with a new method of compound scaling that scales the width of the network, the depth of the network, and resolution simultaneously. Its architecture has been able to outperform all the previous networks in several image classification challenges. In terms of deep fake detection,

Khalid et al. [8] used EfficientNet to extract features in their ensemble model for deep fake detection and reported suc979-8-3315-1795-3/24/\$31.00©2024 IEEE

3) *cess*.

4) *Inception*: The architecture called Inception, was first proposed by Szegedy et al. [9], and focuses on maximizing the use of the computing resources within the network. It employs convolutional layers with parallel branches that have different kernel sizes, enabling the model to learn features at different resolution levels at the same time. An Inception-based network was used by Amerini et al. [10] for manipulated face detection in pictures and videos which proved to efficiently capture details of fine granularity necessary for deepfake detection.

5) *Xception*: Xception, proposed by Chollet [11], wisely uses depth-wise separable convolutions instead of the Inception modules, which has made the model reap its full benefits. In the domain of Deepfake detection, Rossler et al. [2] in the FaceForensics++ benchmark, discovered that Xception outperformed its peers in several types of facial deformation detection tasks. Next, Hsu et al. [12] modified the Xception model also for deepfake detection and introduced the attention mechanism to improve performance on hard datasets.

C. Recurrent Neural Networks for Temporal Analysis

On the one hand, it is known that Convolutional Neural Networks (CNN) are very effective in extracting spatial features, however, Recurrent Neural Networks (RNNs) are better able to understand the relationships among successive video frames. One of them is Gated Recurrent Units which were introduced by Cho et al. [13] and have appeared in practice owing to their efficiency for the same ones featuring long-term relations. Sabir et al. [14] combined CNN with recurrent networks to deal with temporal inconsistencies for deepfake detection. Their method gave full evidence of the opportunity to use both types of information which helped decrease the number of errors.

D. Comparative Studies

Many authors conducted comparative studies between the different deep-learning models or specialized algorithms used for deep-fake detection. In particular, Tolosana et al. [15] aimed to comprehensively assess the performance of several basic CNNs including ResNet and EfficientNet in terms of their effectiveness on various deepfake datasets. Their analysis reaffirmed virtually all the scientists' propositions on the need to determine the right architectures and training approaches for Internal models. Similarly, Bonettini et al. [16] made a comparative study of the best available deepfake detectors with an emphasis on the requirements for effective validation of detection algorithms and datasets that would represent the population of deep-fake videos.

E. Research Gaps

However, despite the achievements, the problem of the detection of deep-fakes is still a big issue. According to Westerlund [17], the creation of deep-fakes is controversial and is linked to an ever-ongoing 'arms race' between the methods of their creation and methods aiming at deep-fake

detection. The author stressed that there is a necessity to develop further more efficient detection technologies to keep up with the development of new-generation deepfakes. Taking into account the increasing popularity of the deep-fake phenomenon, Luo et al. [18] devoted their recent work to the challenges of generalizable deep-fake detection tasks that customers panic about detecting novel manipulation techniques that they haven't seen during training. Their strategy underlines the need to design such models that will address the needs of the market as regards the technology of deep fakes which is an ever-changing trend. In analyzing such diverse models, we hope to advance the current debate on deep-fake detection and offer a basis for further research in the field.

III. METHODOLOGY

This segment describes the steps followed in the detection of deepfakes, including the dataset, the preprocessing steps carried out, the models built and the training carried out.

A. Dataset

The study employed a deep fake detection dataset comprising 12,500 videos comprising of:

- 7500 fake videos (deepfakes)
- 5000 real videos

This dataset is quite imbalanced, showing the real world where targets of this type often have more manipulated content than real content. The study was done by Rossler et al. [19] and quantitatively assessed using the dataset FaceForensics++.

B. Data Preprocessing

To get the data ready for our models, we performed the following preprocessing steps:

- 1) *Frame Extraction*: Each of the videos located in the input folder was handled separately. To save on computation, while retaining enough temporal information for analysis, every third frame was captured for analysis.
- 2) *Face Detection and Cropping*:
 - a) *Frame Resizing*: Every frame chosen was first shrunk to fifty percent of the actual size to help quicken face identification.
 - b) *Face Detection*: The frames obtained were searched for faces with the help of the face_recognition library [20].
 - c) *Coordinate Scaling*: Subsequently, once a frame had a face, the coordinates of the frame were rescaled to the original size of the frame.
 - d) *Standardization*: All cropped faces were cropped to a dimension of 224x224 pixels to achieve uniformity in the entire sample.
- 3) *Video Reconstruction*:
 - a) *Saving Cropped Video*: The cropped ranges of face video frames were arranged in a normal video file which was created while preserving the frame rate of the original video, though the number of frames covered is about 1/3 of that of the original video.
 - b) *Short Video Removal*: A post-process eliminates any videos from the output folder that are less than 60 frames. For most threshold videos, a certain cut-off

was there to further enhance the temporal resolution within the time frame.

4) Quality Control:

- a) *Dimension Verification*: A separate function using moviepy [21] was introduced in order to check the resolution of the processed videos.
- b) *Metadata Creation*: A CSV document was created comprising video names and the corresponding labels (real or fake) for both original and augmented videos.

5) Data Augmentation:

Random vertical flipping and random slight rotation of the input frames were done during the preprocessing as they help enhance the generalization of the model a practice common in computer vision tasks [22].

C. Model Architecture

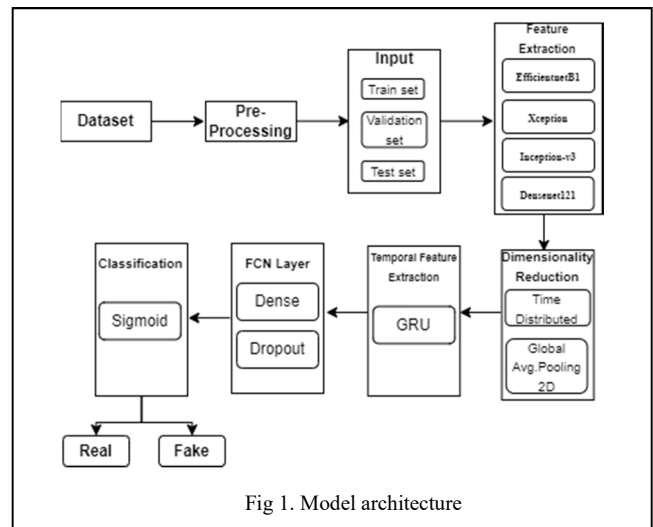


Fig 1. Model architecture

Our method connects four different modern Convolutional Neural Network architectures with a Gated Recurrent Unit to perform analysis in the forward temporal direction. The four CNN models used are:

- DenseNet-121 [23]
- Inception-v3 [24]
- EfficientNet-B1 [25]
- Xception [26]

Fig 1. represents the model architecture.

As for each of these models, we employed the following architecture:

- a) *CNN Base Model*: Concerning each of these models we undertook transfer learning and therefore used these CNNs as feature extractors with pre-trained weights on large datasets of images [27].
- b) *Time Distributed Layer*: This layer performs the task of Convolutional Neural Network (CNN) to every frame which is beneficial in looking at the sequence of frames [28].
- c) *Global Average Pooling 2D*: This is used to reduce the dimensionality of features after CNNs for individual frames [29].

d) *GRU Layer*: A GRU layer of 128 units is employed to process the sequence of frame features, utilizing the temporal information of the video [30].

e) *Dense Layer*: The dense layer of 128 units was brought after the GRU and had activation function swish and dropout layer of value 0.6 and the last dense layer has 1 unit sigmoid activation function for classification of samples as real or fake ontology.

D. Training Process

All four integrated models (CNN + GRU) were developed utilizing the same training process. This standardization of procedures promotes the comparability of results and helps integrate the best practices in machine learning. The process included the following steps:

- 1) *Frame Selection*: Five frames were extracted from video clips, to adequately represent the video content.
- 2) *Frame Resizing*: Each retrieved frame measures 112x112 pixels before being processed into the model.
- 3) *Data Split Ratio*: 70% training 15% validation 15% testing Integrity No single set of frames from a single video shall be split into two different sets. Justification: This split ratio and integrity measure is the current state of practice in machine learning allowing for adequate model validation whilst avoiding data overfitting "in that we can also call it 'data leakage'" [30].
- 4) *Loss Function Choice*: Binary Cross-Entropy Justification: This is a type of loss function used in cases of binary classification problems which is applicable in the current problem of deep fake detection, which is only real vs fake detection [31].
- 5) *Optimizer Choice*: Adam optimizer Learning rate: 0.00001 Justification: As proposed by Kingma and Ba, this configuration provides individual learning rates for each parameter which may improve the rate of convergence of the algorithm [32].
- 6) *Batch Size*: 64 Justification: This batch size was found to allow for an acceptable trade-off between model accuracy and computer resource usage. It allows parallelizing of processing as efficiently as possible while giving the optimization process enough stochasticity [33].
- 7) *Epochs*: Maximum 25 epochs. Early Stopping: Procedure which is used to avoid overfitting. Justification: Early stopping is a conceptual view attributed to overfitting in which the training of the model is stopped on the account that validation set loss has not decreased for some time [34].
- 8) *Class Weighting*: Implementation: Used to solve the dataset imbalance issue. Justification: There were more fake videos when compared to real ones in the dataset. Class weighting serves to reduce the effects of this imbalance, reducing bias in the model's output [35].

9) Additional Training Techniques:

a) *ReduceLROnPlateau*: The learning rate of the model is decreased whenever its performance becomes

constant in order to achieve further adjustment of the model attributes [36].

b) *Model Checkpoint*: This allows the best version of the model to be captured in the course of the training such that there shall be no deterioration of performance due to a dip in performance in the later training stages [37].

By applying these techniques, we set out to build a training process that yielded reliable deepfake detection models that performed well. The uniformity of the procedure in all models enables direct evaluation of the models.

IV. RESULTS

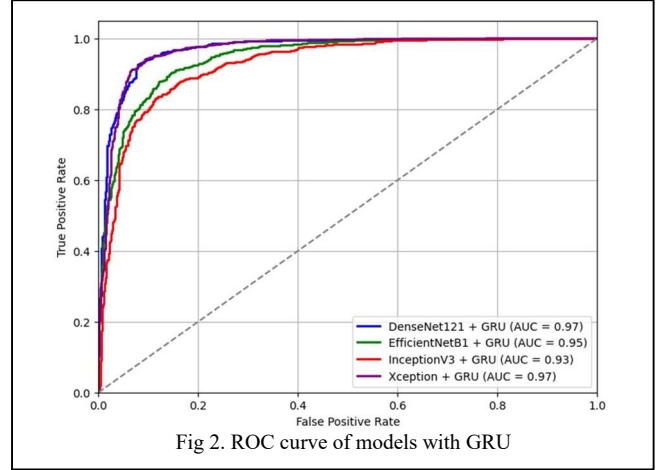


Fig 2. ROC curve of models with GRU

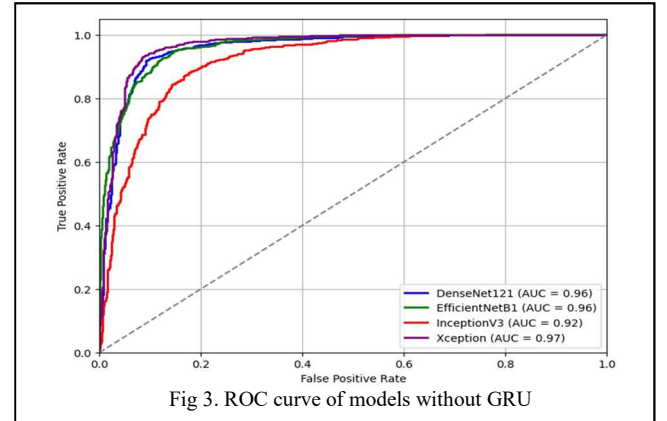


Fig 3. ROC curve of models without GRU

The present study examined the effectiveness of the four leading convolutional neural network architectures in the deepfake detection problem. Each architecture was evaluated not only in its native mode but also with Gated Recurrent Units (GRU) added to evaluate the effect of temporal information processing on detection performance. In this paper, we trained and tested a total of 12,500 videos, this includes 7500 fake and 5000 true videos from the Kaggle deep fake detection challenge data set.

To evaluate and make comparisons with the performance of our developed models, we were based on the values of the following metrics:

- 1) *Overall accuracy*: General correctness of the model
- 2) *Positive predictive value or Precision*: The number of correctly classified instances that are truly positive divided by the total number of positive predictions.
- 3) *Sensitivity or Recall*: Also, the proportion of true positive cases over total actual positive cases.

4) *F1-Score*: It is the weighted average of precision and recall. Receiver Operating Characteristic Curve (ROC) These metrics ensure a thorough appraisal of every model. This is done in terms of considerations of the inclusion of aspects of ELT-ESL in the imbalanced data set as Fawcett [36] suggests. Fig 2 and Fig 3 show the ROC curve and AUC values of the model with GRU and standalone.

A. Model Performance Comparison:

TABLE I. Models Accuracy with GRU

Models	Accuracy	Loss
DenseNet121+GRU	0.9248	0.2146
EfficientNetB1+GRU	0.8795	0.3079
Xception + GRU	0.9237	0.2206
InceptionV3 + GRU	0.8549	0.3381

TABLE II. Confusion Matrix of models with GRU

Models	Real/Fake	Precision	Recall	F1-score
DenseNet121+GRU	Real	0.92	0.89	0.90
	Fake	0.93	0.95	0.94
EfficientNetB1+GRU	Real	0.87	0.82	0.84
	Fake	0.89	0.92	0.90
Xception+GRU	Real	0.92	0.88	0.90
	Fake	0.93	0.95	0.94
InceptionV3+GRU	Real	0.82	0.80	0.81
	Fake	0.87	0.89	0.88

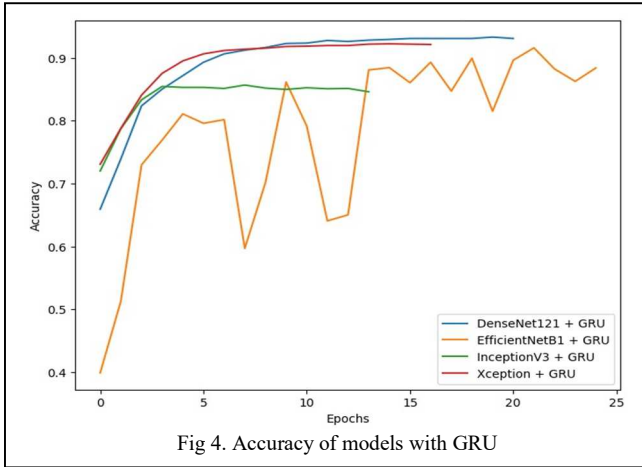


Fig 4. Accuracy of models with GRU

1) *Models with GRU Integration*: The integration of GRU layers with the CNN architectures yielded the following results. The following Table I represents model accuracy and loss of models integrated with GRU. Fig 4 represents the validation accuracy and Table II represents the confusion matrix of models

Among the GRU-enhanced models, the highest relative achievement of classification in DenseNet121+GRU components reached 92.48% accuracy with a minimum loss of 21.46. Xception+GRU performance accuracy was lower – 92.37%, but only 22.06% of the loss was suffered. The performance of EfficientNetB1+GRU and InceptionV3+GRU models were comparatively poorer.

2) Models without GRU Integration:

TABLE III. Accuracy of models without GRU

Models	Accuracy	Loss
DenseNet121	0.9120	0.2598
EfficientNetB1	0.9072	0.3036
Xception	0.9259	0.2301
InceptionV3	0.8587	0.3996

TABLE IV. Confusion matrix of models without GRU

Models	Real/Fake	Precision	Recall	F1-score
DenseNet121	Real	0.90	0.88	0.89
	Fake	0.92	0.94	0.93
EfficientNetB1	Real	0.90	0.86	0.88
	Fake	0.91	0.94	0.92
Xception	Real	0.92	0.89	0.90
	Fake	0.93	0.95	0.94
InceptionV3	Real	0.83	0.81	0.82
	Fake	0.88	0.89	0.88

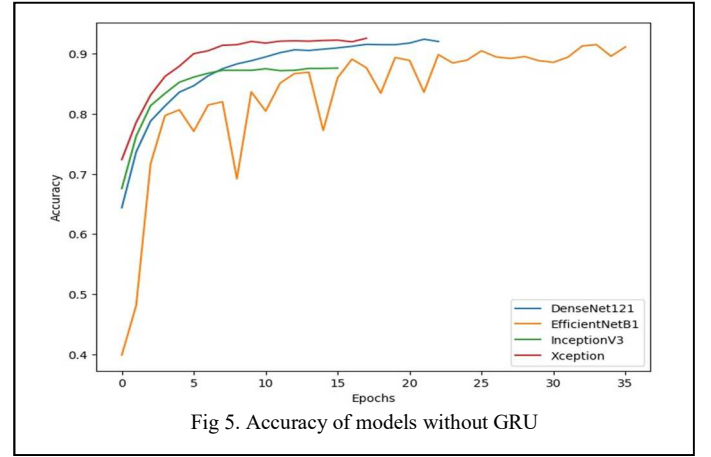


Fig 5. Accuracy of models without GRU

The following Table III represents model accuracy and loss of models standalone. Fig 5 represents the validation accuracy and Table IV represents the confusion matrix of models

The performance of the standard CNN architectures without GRU layers was as follows:

Xception, under this configuration, emerged as the most accurate and efficient at 92.59 % with a 23.01% loss. DenseNet121 and EfficientNetB1 were also able to achieve good accuracy. In contrast, InceptionV3 had the lowest accuracy and the highest loss in all the models tested.

B. Effects of Implementing GRU:

Even though GRU layers were integrated into the different architectures, diverse effects were noted.

1) *DenseNet121*: In the case of GRU integration, accuracy was enhanced by 1.28 percentage points while loss was reduced by 4.52 percentage points.

2) *EfficientNetB1*: The addition of GRU layers was noted to bring down the accuracy rate by 2.77 percentage points although slightly depressed the loss by 0.43 percentage points.

3) *Xception*: The use of GRU integration lowered the accuracy margin as compared to the control base case by 0.22

percentage points however improved the overall loss by 0.95 percentage points.

4) *InceptionV3*: The application of the GRU layers however reduced the accuracy slightly by 0.38 percentage points while the improvement in loss was 6.15 percentage points.

V. CONCLUSION

This work includes an extensive performance appraisal of four popular models: DenseNet121, EfficientNetB1, Xception, and InceptionV3 for deepfake detection with or without Gated Recurrent Units (GRU). By employing a dataset of 12,500 videos (7500 fake and 5000 real) from the Kaggle Deep Fake Detection Challenge our research has reached and produced several findings:

- 1) Xception and DenseNet121 outperformed other architectures on those tasks, Xception without GRU obtained the highest accuracy of 92.59%, while DenseNet121 with GRU obtained 92.48% and the second place overall.
- 2) The addition of GRU layers produced mixed results across the different architectures. The model architecture of DenseNet121 showed the greatest benefit from the GRU integration with an increase in accuracy by 1.28 percentage points, while most of the other models showed negligible effects or some negative effects.
- 3) The balanced nature of the dataset helped achieve high performance for all models and in any machine learning task which highlights the need for adequacy in the datasets and the diversity in the deepfake detection domain.
- 4) It can be concluded from this that quite a few networks perform similarly concerning GRU and without them pointing out that for many real-time situations lightweight architectures may be more attractive.

These resources help us understand how various CNN structures work in deepfake detection, as well as shed light on the temporal aspect of this problem. Regardless of how sophisticated deepfake technology becomes, our findings are relevant to the current trend of building strong qualitative deepfake detection systems, thus helping to combat the ubiquitous nature of digital fraud.

VI. FUTURE WORK

Regarding the results of our research paper about deepfakes classification employing different CNN models along with GRU(Dense Block)+ GRU Support and New avenues of investigation we recommend including the following three issues:

Assessment of new temporal processing models

However, as a part of our study, we did add those GRU layers for capturing temporal aspects of modeling but it was not the case for many of the other architectures where it worked well. Towards this end, future studies must explore more effective techniques for integrating temporal aspects in the deepfake detection models. Some of the potential strategies include:

- Using 3D Convolutional Neural Networks (3D CNNs) to seizures the video sequences which consist of both spatial and temporal characteristics.

- Using attention mechanisms, such as self-attention or temporal attention, in order to highlight the spatial or temporal frames/features most salient to detecting deepfakes.
- Building hybrid neural architectures by incorporating Convolutional Neural Networks (CNNs) with more sophisticated recurrent types of architectures such as Long short-term memory networks (LSTM networks) or Transformer models.

Such approaches may also potentially overcome the shortcomings of the existing deep fake detection and protection systems, especially regarding capturing subtle temporal dissimilarities in sequences.

Dealing With Generalisation and Transferability Issues Raised by New Deepfake Techniques

In order to respond to the evolution of deepfake technology, the detection models must evolve and are able to withstand the new forms of manipulation. It is therefore important to say that the future of deepfake detection technologies is in expanding the scope within which their generalisation abilities fit. Some of the important areas in this regard include:

- Come up with strategies and models of continual learning for the models allowing them to learn new types of deepfakes while still being able to utilize the ones already in memory.
- Learning how to cope with different complete video qualities, resolutions, and compression ratios, through the use of domain adaptation techniques.
- Training and assessing more effective anti-deepfake models by creating a variety of new datasets with emerging techniques of deepfake synthesis.
- Making use of passively collected unlabeled video data, without the supervision of experts, is aimed at improving the generalization of deepfake detection models through unsupervised or self-supervised learning paradigms.

By working on these issues, subsequent work aims to develop more efficient and future-proof detection systems able to keep pace with the ever-changing picture of deepfake images.

REFERENCES

- [1] Y.Li, M.C. Chabf, and S.Lyu, "In ictu oculi:Ecposing AI generated fake face videos by detecting ehe blinking," in IEEE International Workshop on Information Forensics and Security(WIFS), 2018, pp. 1-7.
- [2] A. Rossler,D.Cozzolino,L.Verdoliva, C.Riess, J.Thies, and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images," in IEEE/CVF International Conference on Computer Vision (ICCV),2019,pp. 1-11.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.K. Elissa, "Title of paper if known," unpublished.
- [4] Nguyen, H. H., Yamagishi, J., & Echizen, I. (2019). Capsule-forensics: Using capsule networks to detect forged images and videos. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2307-2311.M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.
- [5] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.

- [6] Trinh, L., Tsang, M., Rambhatla, S., & Liu, Y. (2021). Interpretable and trustworthy deepfake detection via dynamic prototypes. *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 1973-1983.
- [7] Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning (ICML)*, 6105-6114.
- [8] Khalid, H., & Woo, S. S. (2020). OC-FakeDect: Classifying deepfakes using one-class variational autoencoder. *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 656-657.
- [9] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1-9).
- [10] Amerini, I., Galteri, L., Caldelli, R., & Del Bimbo, A. (2019). Deepfake video detection through optical flow based CNN. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*.
- [11] Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1251-1258).
- [12] Hsu, C. C., Lee, C. Y., & Zhuang, Y. X. (2020). Learning to detect fake face images in the wild. In *2020 International Symposium on Computer, Consumer and Control (IS3C)* (pp. 388-391). IEEE.
- [13] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724-1734.
- [14] Sabir, E., Cheng, J., Jaiswal, A., AbdAlmageed, W., Masi, I., & Natarajan, P. (2019). Recurrent convolutional strategies for face manipulation detection in videos. *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 80-87.
- [15] Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). DeepFakes and beyond: A survey of face manipulation and fake detection. *Information Fusion*, 64, 131-148.
- [16] Bonettini, N., Cannas, E. D., Mandelli, S., Bondi, L., Bestagini, P., & Tubaro, S. (2021). Video face manipulation detection through ensemble of CNNs. *International Conference on Pattern Recognition (ICPR)*, 5012-5019.
- [17] Westerlund, M. (2019). The emergence of deepfake technology: A review. *Technology Innovation Management Review*, 9(11), 39-52.
- [18] Luo, Y., Zhang, Y., Yan, J., & Liu, W. (2021). Generalizing face forgery detection with high-frequency features. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16317-16326.
- [19] A. Rossler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1-11.
- [20] Geitgey, A. (2017). *face_recognition*: The world's simplest facial recognition api for Python and the command line. GitHub repository. https://github.com/ageitgey/face_recognition.
- [21] Zulko. (2017). *MoviePy*: Video editing with Python. GitHub repository. <https://github.com/Zulko/moviepy>
- [22] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861-874, 2006.
- [23] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700-4708.
- [24] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818-2826.
- [25] Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning* (pp. 6105-6114). PMLR.
- [26] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2818-2826).
- [27] J. Donahue et al., "Long-term recurrent convolutional networks for visual recognition and description," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 2625-2634.
- [28] M. Lin, Q. Chen, and S. Yan, "Network in network," *arXiv preprint arXiv:1312.4400*, 2013.
- [29] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2014, pp. 1724-1734.
- [30] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. New York, NY, USA: Springer, 2009.
- [31] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT press.
- [32] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- [33] Masters, D., & Luschi, C. (2018). Revisiting small batch training for deep neural networks. *arXiv preprint arXiv:1804.07612*.
- [34] Prechelt, L. (1998). Early stopping-but when?. In *Neural Networks: Tricks of the trade* (pp. 55-69). Springer, Berlin, Heidelberg.
- [35] He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering*, 21(9), 1263-1284.
- [36] Smith, L. N. (2017). Cyclical learning rates for training neural networks. In *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)* (pp. 464-472). IEEE.
- [37] Chollet, F. (2015). *Keras*. GitHub. <https://github.com/fchollet/keras>